成果物作成期間	2026/1/5 ~ 2026/3/18
ポートフォリオ作成日	2026/3/18
所要時間	176時間
協力者	なし
タイトル	福岡市のごみ分別AI
テーマ選定の理由	自身でごみを分別する際に、適切な区分が分からず都度調べる必要があるといった課題を感じた。この課題に対し、画像識別技術を活用することで、ごみの種類を自動判別し分別を支援できるのではないかと考え、本テーマを設定した。
想定業界	福岡市等のごみを収集している自治体・団体
想定企業	福岡市環境局
想定ポジション	循環型社会推進部ごみ減量推進課
背景	1.収集側の課題 環境省の統計によると、ごみ処理にかかる費用は年々増加しており、2023年度には約2.3兆円に達している（参考文献①）。この費用には、焼却施設の維持管理費、収集・運搬費、施設の建設・改修費などが含まれており、その多くは税金やごみ処理手数料によって賄われている。 また、リサイクル率の低さも課題となっている。日本のリサイクル率は近年20％前後で推移しており、ヨーロッパ諸国と比較しても低い水準にある。 さらに、鎌倉市の調査によると、燃えるごみとして廃棄されたもののうち約22％が本来リサイクル可能な資源ごみであったと報告されている（参考文献③）。 このように、ごみの分別が適切に行われていない現状が、リサイクル率の低下や焼却量の増加につながっていると考えられる。分別の精度が向上すれば、資源回収量の増加および焼却量の削減を通じて、将来的な処理コストの削減が期待される。 2.廃棄側の課題 一方で、家庭ごみを排出する立場においても、分別に関する課題が存在する。 主な要因としては、以下が挙げられる。 ・分別ルールが複雑である ・品目名称が分かりにくい ・自治体ごとにルールが異なる 実際に、株式会社オレンジページの調査によると、「分別の判断に困るごみがある」と回答した人は全体の45.9％に上る（参考文献④）。 これらの課題により、本来リサイクル可能な資源ごみであっても適切に分別されず、可燃ごみとして廃棄されるケースが発生している。その結果、ごみの焼却量の増加や処理コストの上昇につながっていると考えられる。
目的	1.収集側 本成果物を用いてごみ分別を自動化することにより、「古紙として適切に分別・廃棄すれば再利用できるが、誤って燃えるごみとして廃棄してしまったために焼却されてしまう」などのご分類によるごみの焼却費用の増加を防ぎ、リサイクル率の向上につなげることができる。 2.廃棄側 ごみの分類を調べる際、福岡市のHP内を検索するのに数十秒～１分程度時間がかかってしまっていた。 福岡市のLINEと本成果物を連携させることにより、この作業工程を　写真撮影→送信　に置き換えることにより、かかる時間を数秒にまで短縮することができる。
概要	ResNetモデルを用いて廃棄物の画像からそれがどのごみ区分（燃えるごみなど） に分類されるのかを予測するAI
アプローチの理由	ResNetは残差接続（Residual Connection）を導入することで勾配消失問題を抑制し、 深いネットワークでも安定した学習が可能なモデルである。 また、ImageNetで事前学習された重みを活用できるため、 少ないデータ数でも高い特徴抽出能力を期待できる。 今回の成果物ではデータを自力で収集したため、集めらるデータに限りがあった。 そこでResNetモデルならば上記の特徴により高い予測精度が期待できると考え採用した。

結果を何に活かせるのか	本成果物は、入力された画像データに対して分類結果を出力する画像認識モデルである。 本研究では「ごみ分類」を対象とし、廃棄物に分類される品目の画像データを用いて学習を行ったが、学習データを別の対象に置き換えることで、様々な分類タスクに応用することが可能である。 例えば、製造業においては、正常品および不良品の画像を学習させることで、製品の外観検査や不良品検出の自動化に応用できる。これにより、検品作業の効率化や品質の安定化が期待される。 また、本成果物では転移学習を採用している。転移学習は、既存の学習済みモデルの特徴抽出能力を活用することで、比較的少ないデータでも高い性能を発揮できる手法である。この特性により、医療画像診断、製造業の外観検査、農作物の病害検出など、学習データの収集が困難な分野においても適用可能であり、高い予測精度が期待できる。
使用言語・環境	使用言語：python　使用環境：google colaboratory

成果物（出力結果）	
結果	本成果物では、ごみ分別AIとして表1の2段目以下の64クラスの細分類を対象に学習を行った。 その後、各クラスの予測結果を表1の１段目が示す福岡市のごみ分別区分に基づき再分類し、最終的な出力としている。 モデル全体の学習から予測までの流れを図1に示す。 この評価方法に基づき精度を算出した結果、Accuracyは80.74%となった。
効果	1.時間短縮について 従来、ごみの分別を調べる際には、福岡市のホームページから品目名を検索する必要があり、数十秒から1分程度の時間を要していた。 一方、本成果物を用いる場合は、画像を撮影して送信するのみで判定が可能であり、処理時間を数秒程度まで短縮することができる。 これにより、ごみ分別にかかる手間の大幅な削減が期待できる。 2.予測精度について 狛江市の調査によると、可燃ごみとして廃棄されたもののうち、不適物の割合は11.9%であった（参考文献⑤）。 一方で、本成果物の最終出力における予測精度はAccuracy 80.74%であり、実運用レベルと比較すると、さらなる精度向上の余地があると考えられる。 以上から、本成果物はごみ分別にかかる時間を大幅に短縮できる点で有用である一方、予測精度の観点では改善の余地が残されている。 今後はデータ数の拡充やクラス設計の見直し、モデルの改善を行うことで、実運用に耐えうる精度への向上が期待される。
工夫点・苦労点	. 苦労点 本成果物の開発における課題として、「個人で収集可能な画像データ数に限界がある」点が挙げられる。 その結果、学習データの不足によりモデルの予測精度が伸び悩むという課題が生じた。 2. 工夫点 （1）クラス設計の見直し 当初は「燃えるごみ」「燃えないごみ」などの大分類を直接予測する方式を採用していたが、現在はより細かいクラス（小分類）で学習を行い、その予測結果を最終的なごみ区分へ変換する方式に変更した。 この設計により、福岡市以外の自治体においても、クラスとごみ区分の対応関係（マッピング）を変更するだけで対応可能となり、汎用性の高いシステムを実現している。

	（2）転移学習の最適化 本成果物では、ResNetを用いた転移学習を採用している。 学習にあたっては、低層（浅い層）の重みを凍結し、物体の高次特徴を学習するlayer3およびlayer4のみを学習対象とした。 これにより、ImageNetで事前学習された汎用的な特徴抽出能力を維持しつつ、ごみ画像特有の特徴に適応させることができ、限られたデータ環境においても精度向上を実現した。
今後の施策	今後の改善施策として、物体検出モデルであるYOLOの導入が挙げられる。 現在のモデルは画像全体を単一の対象として分類する方式であるため、1枚の画像に複数のごみが含まれている場合、それぞれを個別に撮影する必要があるという課題がある。 YOLOを導入することで、1枚の画像内に存在する複数のごみを同時に検出・分類することが可能となる。これにより、ユーザーはごみを1つずつ分けて撮影する必要がなくなり、操作の簡略化および利便性の向上が期待される。 さらに、物体ごとの位置情報（バウンディングボックス）を取得できるため、ごみの種類ごとの分布把握や誤分類の分析など、システム改善にも活用可能である。

燃えるゴミ	燃えないゴミ	空き瓶、ペットボトル	粗大ごみ	古紙	小型家電	家電リサイクル法対象品	パソコン
衣類	アルミ製品	空き瓶	椅子	牛乳パック	アナログ時計	エアコン	デスクトップパソコン
紙製品	石、タイル、レンガ	ペットボトル	カーペット、マット	雑誌、本	カメラ	洗濯機	ノートパソコン
CD、DVD	AV機器		ギター	段ボール	携帯。スマホ	TV	
植物	ガラス、陶器		ゴルフバッグ	チラシ、封筒	ゲームコントローラー	冷蔵庫	
生ごみ	金属製容器		スキー板、スノーボード		体温計		
フィギュア	金属部品		ストーブ		デジタル時計		
プラスチック製バケツ	照明器具		棚		電卓		
プラスチック製歯ブラシ	スプレー缶		テーブル		電動歯ブラシ		
プラスチック製ハンガー	扇風機		電子鍵盤		電動髭剃り		
プラスチック製容器	掃除機		電子レンジ		ドライヤー		
ボール	トースター		ドラム		リモコン		
木製製品	ポット		トレーニングマシン				
レゴブロック			乗り物				
			ベッド、ソファ、				
			マットレス				
			ミシン				

表1.今回作成したクラス

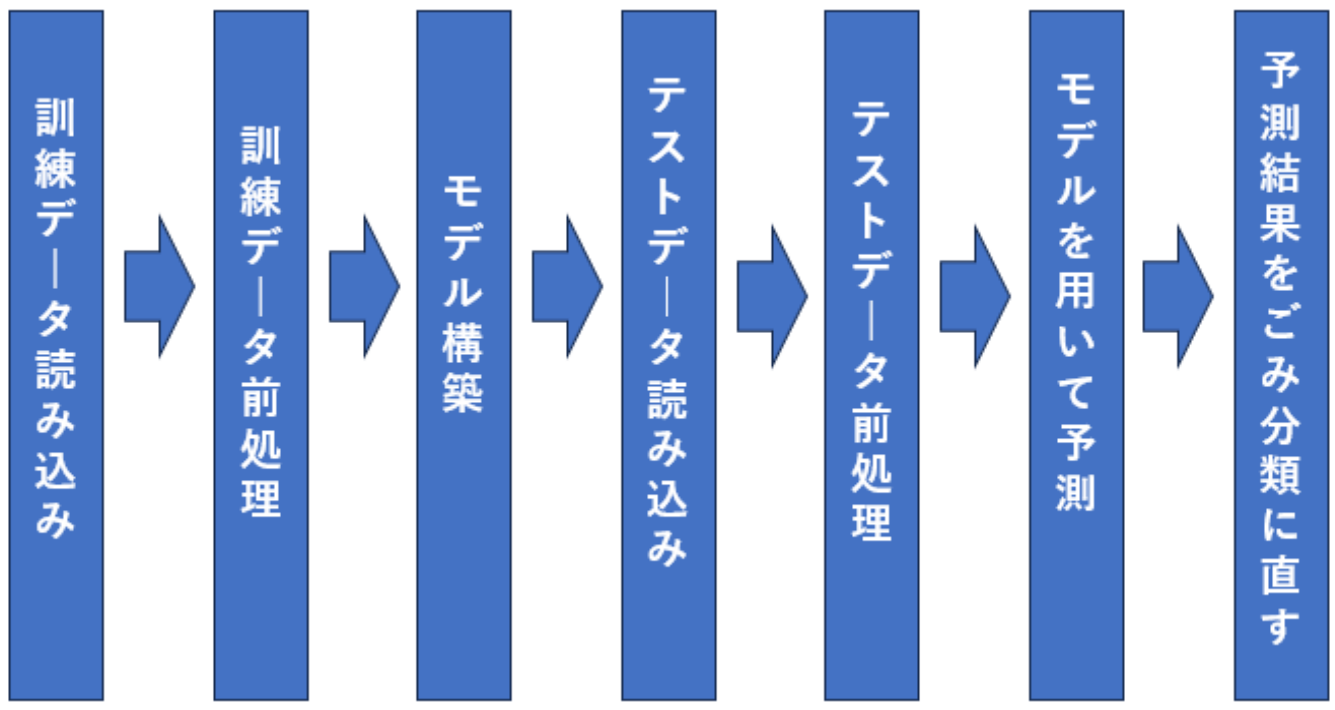

図1.モデル構築～予測までの流れ

記述コード

```
from PIL import Image
import os

root = "/content/drive/MyDrive/ごみ分別AI/garbage_crops3"

for subdir, dirs, files in os.walk(root):
    for file in files:
        if file.lower().endswith((".jpg", ".png", ".jpeg")):
            path = os.path.join(subdir, file)
            img = Image.open(path)
            img = img.convert("RGB")
            img = img.resize((512, 512))

            img.save(path)


import os
import random

BASE_DIR = "/content/drive/MyDrive/ごみ分別AI/garbage_crops3"
MAX_IMAGES = 100

for class_name in os.listdir(BASE_DIR):
    class_path = os.path.join(BASE_DIR, class_name)

    if not os.path.isdir(class_path):
        continue

    images = [
        f for f in os.listdir(class_path)
        if f.lower().endswith((".jpg", ".jpeg", ".png"))
    ]

    print(f"{class_name}: {len(images)} 枚")

    if len(images) > MAX_IMAGES:



        keep_images = random.sample(images, MAX_IMAGES)

        for img in images:
            if img not in keep_images:
                os.remove(os.path.join(class_path, img))

        print(f" → {MAX_IMAGES} 枚")

print("完了")

import os
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models
from sklearn.model_selection import train_test_split

# ========================================
# ① パス設定
```

```
# ==========================================
BASE_DIR = "/content/drive/MyDrive/ごみ分別AI/garbage_crops3"
CHECKPOINT_PATH = "/content/drive/MyDrive/ごみ分別AI/resnet_checkpoint4.pth"
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
EPOCHS = 50
BATCH_SIZE = 32

# ==========================================
# ② データ拡張設定
# ==========================================
train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.6, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(20),
    transforms.ColorJitter(
        brightness=0.3,
        contrast=0.3,
        saturation=0.3,
        hue=0.05
    ),
    transforms.RandomGrayscale(p=0.1),
    transforms.GaussianBlur(kernel_size=3),

    transforms.ToTensor(),

    transforms.RandomErasing(p=0.3),

    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

test_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

# ==========================================
# ③ データセット読み込み
# ==========================================
full_dataset = datasets.ImageFolder(root=BASE_DIR)

train_size = int(0.8 * len(full_dataset))
test_size = len(full_dataset) - train_size

targets = full_dataset.targets

train_indices, test_indices = train_test_split(
    list(range(len(targets))),
    test_size=0.2,
    stratify=targets,
    random_state=42
)

train_dataset = torch.utils.data.Subset(
    datasets.ImageFolder(root=BASE_DIR, transform=train_transforms),
    train_indices
)

test_dataset = torch.utils.data.Subset(
```

```
    datasets.ImageFolder(root=BASE_DIR, transform=test_transforms),
    test_indices
)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)

# ========================================
# ④ モデル構築（ResNet18）
# ========================================
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)


for param in model.parameters():
    param.requires_grad = False


model.fc = nn.Linear(model.fc.in_features, 64)

for param in model.layer2.parameters():
    param.requires_grad = True

for param in model.layer3.parameters():
    param.requires_grad = True

for param in model.layer4.parameters():
    param.requires_grad = True

for param in model.fc.parameters():
    param.requires_grad = True

model = model.to(DEVICE)

# 損失関数（label smoothing）
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

# optimizer
optimizer = torch.optim.Adam([
    {"params": model.layer3.parameters(), "lr": 1e-5},
    {"params": model.layer4.parameters(), "lr": 5e-5},
    {"params": model.fc.parameters(), "lr": 1e-4},
])

# CosineAnnealing
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=EPOCHS,
    eta_min=1e-6
)

# ========================================
# ⑤ 学習ループ
# ========================================

start_epoch = 0

if os.path.exists(CHECKPOINT_PATH):
    print(" checkpoint読み込み")

    checkpoint = torch.load(CHECKPOINT_PATH)

    model.load_state_dict(checkpoint["model_state"])
    optimizer.load_state_dict(checkpoint["optimizer_state"])
    scheduler.load_state_dict(checkpoint["scheduler_state"])
```

```
    start_epoch = checkpoint["epoch"] + 1

    print(f"再開epoch: {start_epoch}")

for epoch in range(start_epoch, EPOCHS):

    model.train()
    total_loss = 0

    for images, labels in train_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)

        optimizer.zero_grad()
        outputs = model(images)

        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    scheduler.step()

    print(f"Epoch [{epoch+1}/{EPOCHS}] - "
          f"Train Loss: {total_loss/len(train_loader):.4f}")

    # checkpoint保存
    torch.save({
        "epoch": epoch,
        "model_state": model.state_dict(),
        "optimizer_state": optimizer.state_dict(),
        "scheduler_state": scheduler.state_dict()
    }, CHECKPOINT_PATH)

    print("checkpoint保存")
# ========================================
# ⑥ 評価
# ========================================
model.eval()
correct, total = 0, 0

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(DEVICE), labels.to(DEVICE)

        outputs = model(images)
        _, preds = torch.max(outputs, 1)

        total += labels.size(0)
        correct += (preds == labels).sum().item()

print(f"¥n Test Accuracy: {100 * correct / total:.2f}%")

# ========================================
# ⑦ モデル保存
# ========================================
import torch
from torchvision import models
from torch import nn
import numpy as np
from torch import nn, optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms, models
```

```
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import confusion_matrix
from collections import defaultdict
import os
from sklearn.model_selection import train_test_split

EPOCHS = 50
BATCH_SIZE = 32

BASE_DIR = "/content/drive/MyDrive/ごみ分別AI/garbage crops3"

small_to_large = {
"clothes fabric"       : "burnable",
"toys sports"          : "burnable",
"paper items"          : "burnable",
"discs"                : "burnable",
"plants"               : "burnable",
"food_waste"           : "burnable",
"figure"               : "burnable",
"plastic baket"        : "burnable",
"plastic brash"        : "burnable",
"plastic hanger"       : "burnable",
"plastic container"    : "burnable",
"ball"                 : "burnable",
"wood items"           : "burnable",
"LEGO"                 : "burnable",
"alminium foils"       : "non_burnable",
"stone bricks"         : "non_burnable",
"AV device"            : "non_burnable",
"glass ceramic"        : "non_burnable",
"metal container"      : "non_burnable",
"metal frame parts"    : "non_burnable",
"lighting"             : "non_burnable",
"spray can"            : "non_burnable",
"fan"                  : "non_burnable",
"vacuum cleaner"       : "non_burnable",
"toaster"              : "non_burnable",
"pot"                  : "non_burnable",
"glass bottle"         : "bottle",
"plastic bottle"       : "bottle",
"chair"                : "bulky_garbage",
"carpet mat"           : "bulky_garbage",
"guitar"               : "bulky_garbage",
"sports bag"           : "bulky_garbage",
"sports board"         : "bulky_garbage",
"stove"                : "bulky_garbage",
"shelf"                : "bulky_garbage",
"table"                : "bulky_garbage",
"micro wave"           : "bulky_garbage",
"sports machine"       : "bulky_garbage",
"mobility"             : "bulky_garbage",
"bed sofa"             : "bulky_garbage",
"mattress"             : "bulky_garbage",
"sewing machine"       : "bulky_garbage",
"carton"               : "used_paper",
"magazine"             : "used_paper",
"card board"           : "used_paper",
"Used paper misc"      : "used_paper",
"analog clock"         : "small_appliance",
"camera"               : "small_appliance",
"mobile phone"         : "small_appliance",
"game controler"       : "small_appliance",
"thermometor"          : "small_appliance",
"digital clock"        : "small_appliance",
"calculator"           : "small_appliance",
```

```
"electric brash"       : "small_appliance",
"electric shaving"     : "small_appliance",
"hair dryer"           : "small_appliance",
"remote controler"     : "small_appliance",
"air conditioner"      : "recyclable_appliance",
"washer"               : "recyclable_appliance",
"tv"                   : "recyclable_appliance",
"refrigerator"         : "recyclable_appliance",
"desktop pc"           : "PC",
"laptop pc"            : "PC"
}


class_names = CLASS_NAMES

correct_large = 0
total = 0

train_transforms = transforms.Compose([
    transforms.RandomResizedCrop(224, scale=(0.6, 1.0)),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomRotation(20),
    transforms.ColorJitter(
        brightness=0.3,
        contrast=0.3,
        saturation=0.3,
        hue=0.05
    ),
    transforms.RandomGrayscale(p=0.1),
    transforms.GaussianBlur(kernel_size=3),

    transforms.ToTensor(),

    transforms.RandomErasing(p=0.3),

    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

test_transforms = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    ),
])

full_dataset = datasets.ImageFolder(root=BASE_DIR)

train_size = int(0.8 * len(full_dataset))
test_size = len(full_dataset) - train_size

targets = full_dataset.targets

train_indices, test_indices = train_test_split(
    list(range(len(targets))),
    test_size=0.2,
    stratify=targets,
    random_state=42
)
```

```
train_dataset = torch.utils.data.Subset(
    datasets.ImageFolder(root=BASE_DIR, transform=train_transforms),
    train_indices
)

test_dataset = torch.utils.data.Subset(
    datasets.ImageFolder(root=BASE_DIR, transform=test_transforms),
    test_indices
)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False, num_workers=2)


DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

model = models.resnet18(pretrained=False)
model.fc = nn.Linear(model.fc.in_features, len(CLASS_NAMES))

model.load_state_dict(
    torch.load("/content/drive/MyDrive/ごみ分別AI/resnet18_cosine_model.pth",
               map_location=DEVICE)
)

model = model.to(DEVICE)
model.eval()

model.eval()
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(DEVICE)
        outputs = model(images)
        _, preds = torch.max(outputs, 1)

        for true_label, pred_label in zip(labels, preds.cpu()):
            true_small = class_names[true_label]
            pred_small = class_names[pred_label]

            true_large = small_to_large[true_small]
            pred_large = small_to_large[pred_small]

            if true_large == pred_large:
                correct_large += 1

            total += 1

print(f"Large Category Accuracy: {100 * correct_large / total:.2f}%")
```

参考文献
参考文献① 環境省の統計 https://www.env.go.jp/press/press_02960.html 参考文献② 日本と海外のリサイクル率比較 https://www-cycle.nies.go.jp/magazine/kenkyu/202008.html) 参考文献③ 燃えるごみに含まれていた資源ごみの割合 https://www.city.kamakura.kanagawa.jp/gomi/sosei.html?utm_source=chatgpt.com 参考文献④ 株式会社オレンジページの調査結果 https://www.atpress.ne.jp/news/191007 参考文献⑤ 狛江市調査結果 https://www.city.komae.tokyo.jp/index.cfm/51,140741,481,html